MVP PLAN · CLOSED BETA

ADHD Budgeting App

A calm budgeting app for adults with ADHD: one safe-to-spend number, two-tap logging and a guilt-free fresh start after any break.

MVP Build Plan and Technical Requirements
For stakeholders and investors

Inside

1. **Overview:** the opportunity at a glance
2. **MVP Build Plan:** goals, priorities, timeline, design schedule, roles, risks and decisions needed
3. **Technical Requirements:** how the system fits together, technology choices and numbered requirements

October 2026 · Confidential: please do not forward without permission

OVERVIEW

The opportunity at a glance

~366M
adults worldwide have ADHD (about 1 in 14)

14 weeks
to build the MVP, in seven usable phases

100
testers in a 12-week closed beta from week 15

40%
target: testers still active in their fourth week

The problem

Budgeting apps assume steady attention, planning and daily habits, which ADHD makes hard. They fail in four ways: too much setup, balances hidden behind taps, daily logging that breaks after a missed day, and red numbers and broken streaks that shame people into quitting.

Our answer

- One **safe-to-spend** number, always first on screen
- Expenses logged in **two taps**, with undo
- Bills counting down before they hit
- A one-tap **fresh start** after any break, with no guilt

Who it is for

- **Overwhelmed restarters:** have quit 2 or more budgeting apps within weeks
- **Surprise-hit spenders:** keep getting caught by forgotten bills, subscriptions and impulse buys

Why we are different

Money apps (YNAB, Monarch, Rocket Money, PocketGuard, Copilot, EveryDollar) have the tools but not ADHD-first design. ADHD apps (Tiimo, Inflow) have the design but no money features. Our edge is recovery after a lapse, the purchase pause and calm design.

What the beta must prove

Key result	Target
People keep using it	40% of testers active in week 4
People come back after a break	60% of testers who stop for 7+ days return within 30 days
Fewer money surprises	30% fewer late fees or overdrafts by month 3

Built to be trusted

Money data is encrypted on the phone from day one, the app locks itself after 5 minutes, amounts never show on the lock screen unless the person chooses, and bank details never reach the phone. Security is built first and reviewed again before any tester is invited.

PART 1

MVP Build Plan

ADHD Budgeting App: MVP Build Plan

October 26, 2026 · Live version: Claude Doc · Companion: Technical Requirements Document

Summary

We are building the first version (the MVP) of a budgeting app for adults with ADHD. It answers one question at a glance, **"How much can I safely spend right now?"**, and welcomes people back after a break without guilt.

This plan explains **what we build, in what order, who does what, and what could go wrong**. It is written for stakeholders, designers and developers alike; no coding knowledge is needed. The companion *Technical Requirements Document* covers how the app is built.

- **Who it is for:** *overwhelmed restarters* (people who have quit 2 or more budgeting apps) and *surprise-hit spenders* (people caught out by forgotten bills and impulse buys).
- **How long:** about **14 weeks of building** in seven phases, then a **12-week closed beta** with about 100 testers.
- **What success looks like:** 40% of beta testers still use the app in their fourth week.
- **Our biggest bet:** *manual mode first*. People can use the whole app without linking a bank; bank linking comes last.
- **Not negotiable:** money data is protected from day one, before any feature is built.

Words used in this plan

Term	What it means
MVP	Minimum viable product: the smallest version that tests whether the idea works for real people
Closed beta	An invite-only test with about 100 people before the public launch
Safe-to-spend	Your balance minus bills due before payday and money set aside for savings. The app's main number
Buckets	Money shown as three containers that fill and empty: Spending, Bills and Savings
Manual mode	Using the app by typing your balance and expenses, without linking a bank
Bank linking	Connecting a bank account through Plaid so transactions arrive on their own
Phase	A block of work that ends with something testers can actually use
Gate	The check a phase must pass before the next one starts
Development build	A test version of the app installed on a real phone. Needed because of the security features

Term	What it means
Encryption	Scrambling stored data so only the app, unlocked by its owner, can read it
Four screen states	Every screen is designed for *Initial* (empty), *Loading*, *Success* and *Error*
Saved once	A tap that is repeated, or a save retried after a dropped connection, still counts only once

Goals and how we measure success

The beta exists to answer one question: **does an ADHD-first design keep people budgeting, and bring them back after they lapse?**

Goal	Target	How we measure it	When
People keep using it	40% of testers active in week 4	App opens per tester per week (no money details recorded)	Beta week 4
People come back after a break	60% of testers who stop for 7+ days return within 30 days	Return visits after a 7-day gap	Beta weeks 4 to 12
Setup is quick	Setup finished in 5 minutes or less	Time from first open to the home screen	Every new tester
Fewer money surprises	30% fewer late fees or overdrafts	Survey at the start and in month 3	Beta month 3
Built safely	Zero critical security issues open at beta launch	Security checklist and review	Week 14

We record *events* ("opened the app", "finished setup"), never amounts, merchants or balances.

What goes in first

All features below are in the MVP scope, but they are not equally valuable or equally hard. We scored each one so we build the most valuable first and know what to cut if we fall behind.

How we scored: *Impact* (1 to 5: how much it moves the goals above) × *Confidence* (how sure we are, from our research) ÷ *Effort* (estimated weeks). Higher score = build sooner. These are team estimates; we revisit them after the design tests in Phase 0.

Feature	Main goal it serves	Impact	Confidence	Effort (weeks)	Score
Safe-to-spend number with breakdown	Keep using it; fewer surprises	5	90%	1	**4.5**
Quick add with undo	Keep using it	4	80%	1	**3.2**
Bill countdown	Fewer surprises	4	80%	1	**3.2**
Fresh start after a break	Come back after a break	5	60%	1	**3.0**
Step progress and autosave in setup	Setup is quick	4	70%	1	**2.8**
Snoozable reminders	Fewer surprises	4	60%	1	2.4

Feature	Main goal it serves	Impact	Confidence	Effort (weeks)	Score
Goal picker	Setup is quick	2	60%	0.5	2.4
Visual buckets	Keep using it	3	70%	1	2.1
Default categories	Setup is quick	2	50%	0.5	2.0
Weekly check-in	Come back after a break	3	60%	1	1.8
Focus Mode	Keep using it	3	50%	1	1.5
Pause before purchase	Fewer surprises	3	40%	1	1.2
Home-screen widget	Keep using it	4	50%	2	1.0
Recurring bill detection	Setup is quick	3	50%	1.5	1.0
Bank linking and transaction import	Keep using it	4	60%	3	0.8

The **security foundation** is not scored: it is required before anything else ships.

Top 5

1. **Safe-to-spend number.** The core promise of the app. Without it there is nothing to test.
2. **Quick add with undo.** Logging has to take two taps or it will not happen; undo removes the fear of mistakes.
3. **Bill countdown.** Directly targets surprise-hit spenders, one of our two audiences.
4. **Fresh start.** Our clearest difference from competitors. Confidence is lower because no app does it yet, which is exactly why the beta must test it.
5. **Step progress and autosave.** Setup is where ADHD users quit other apps; saving their place protects the first five minutes.

Trade-offs we made

- **Bank linking scores lowest** because it is the most work and depends on an outside company (Plaid). Manual mode tests the core idea without it, so it is built last.
- **The widget** fights "out of sight, out of mind", but needs separate iPhone and Android work plus privacy masking, so it comes late.
- **Visual buckets score in the middle** but are built early, together with safe-to-spend, because they display the same numbers and set the home screen design.

If we fall behind, cut in this order

1. Pause before purchase
2. Home-screen widget
3. Recurring bill detection
4. Bank linking (moves to a beta update; the beta runs in manual mode)

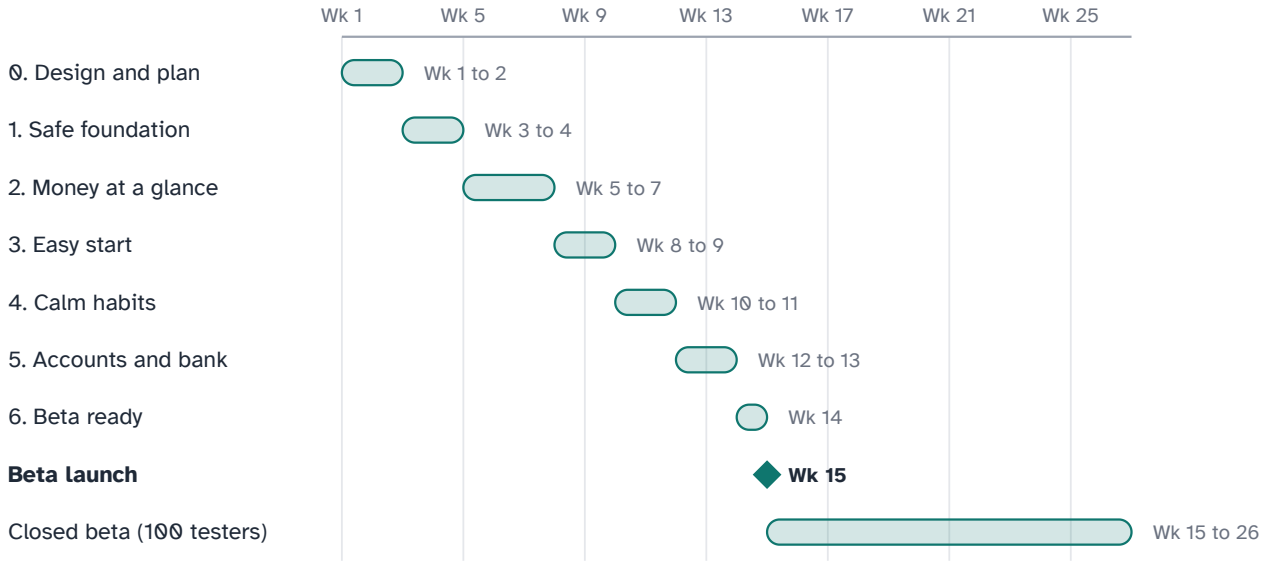
The top 5, the buckets and the security foundation are never cut.

Build phases and timeline

We build in **phases**. Each phase adds one complete, usable piece of the app, so stakeholders can try real progress every two weeks instead of waiting until the end.

Fourteen weeks of building, then a twelve-week closed beta

Estimated weeks for a small team; every phase ends with something testers can use



The axis counts weeks elapsed (0 = the start of week 1). Security comes first, then the everyday money tools, then setup, then habits, and bank linking last.

Phase	Weeks	What we build	What a tester can do afterwards	Gate to move on
0. Design and plan	1 to 2	Mockups, design system, clickable prototype	Click through setup and home in a prototype	6 of 8 test users finish prototype setup in 5 minutes
1. Safe foundation	3 to 4	App shell, encrypted storage, Face ID or fingerprint unlock, 5-minute lock, privacy blur, rooted-phone check	Install the app, unlock it with their face, see an empty home screen	Security checklist passes
2. Money at a glance	5 to 7	Balance, safe-to-spend with breakdown, three buckets, quick add with undo, bills with countdown	Type a balance, add bills, log spending and see the number change	The team uses it daily for a week (internal alpha); all money-math tests pass
3. Easy start	8 to 9	Goal picker, 4-step setup with autosave, default categories	A brand-new person sets up the app alone	5 new people finish setup in 5 minutes or less
4. Calm habits	10 to 11	Fresh start, weekly check-in, snoozable reminders, Focus Mode, pause before purchase	Come back after a week away and restart in one tap	Every screen passes the ADHD design checklist
5. Accounts and bank	12 to 13	Sign-in and backup, bank linking (test banks), transaction import, bill detection, widget	Link a test bank and see transactions sort themselves	Nothing is lost or doubled when the connection drops mid-save

Phase	Weeks	What we build	What a tester can do afterwards	Gate to move on
6. Beta ready	14	Accessibility pass, security review, test builds in TestFlight and Google Play, invite testers	Install the beta from an invite	No critical bugs open

Note on dates: the PRD planned the beta for months 2 to 4. With the full scope, this plan starts the beta in **week 15 (early month 4)**. See *Decisions needed* below.

What design delivers and when

Design stays about one phase ahead of building, so developers never wait for screens. Every screen is delivered in its **four states** (Initial, Loading, Success, Error), in light and dark mode.

Needed by	Design delivers	Why it matters
Week 2	Design tokens (colours, type, spacing), core components (button, money card, bucket, helper text), clickable prototype of setup and home	Sets the look once so every screen stays consistent; the prototype is tested with 8 people at the Phase 0 gate
Week 3	Lock screen, privacy cover shown in the app switcher, "this phone is not supported" screen	Security screens must feel calm, not alarming
Week 5	Home with safe-to-spend and buckets, breakdown, quick add sheet, bills list	The heart of the app
Week 8	Goal picker, 4-step setup with progress, category icons	First impressions decide whether people stay
Week 10	Fresh start, weekly check-in, reminder settings and wording, Focus Mode, pause prompt; short success animations (under 1 second) plus still versions for reduced motion	The habits that bring people back
Week 12	Home-screen widget ("Tap to view" by default), bank-linking explainer, settings	Privacy choices must be clear and easy to change

Design rules for every screen: one decision per screen; no red alarms or shaming words; text at least 16 point in Atkinson Hyperlegible or Lato; buttons at least 44 × 44 point; colour always paired with a word (teal Spending, indigo Bills); amounts hidden outside the app unless the user chooses to show them.

Who does what

Role	Responsible for	Decides
Product manager	This plan, scope, priorities, beta recruiting, success metrics, phase gate reviews	What is built next and what gets cut
Product designer	User flows, mockups, design system, usability tests, wording (with the PM)	How each screen looks and reads

Role	Responsible for	Decides
Mobile developer(s), working with Claude Code	Building each phase, automated tests, test builds on real phones	How it is built, within the technical requirements
Security and data reviewer	Signing off the security checklist at Phase 1 and Phase 6	Whether a phase is safe to release
Instructor and stakeholders	Reviewing each phase demo; approving scope or date changes	Go or no-go at each gate
Beta testers (adults with ADHD)	Using the app, short surveys, optional interviews	What actually helps them
ADHD advisor (optional)	Reviewing wording for shame-free, clear language	Whether copy fits our audience

Risks: a pre-mortem

A pre-mortem imagines the project has already failed and asks why. Picture this: *it is four weeks into the beta and almost nobody is using the app*. What went wrong?

We sort each answer into three kinds:

- **Tigers:** real risks we can see clearly. They need action.
- **Paper tigers:** worries that sound serious but we believe are overblown.
- **Elephants:** things nobody is talking about that might matter.

Tigers

#	Risk	Why we think it is real	Urgency	What we do about it
T1	Too much scope for the time	The full scope needs about 14 weeks; the PRD hoped for a beta in months 2 to 4	**Launch-blocking**	Agree the cut order now and check it at every gate
T2	Not enough testers with ADHD	We need about 100 active testers and have no waitlist yet	**Launch-blocking**	Start recruiting in week 6; aim for 150 sign-ups, about half of them women
T3	Security work slows the team	Encryption, biometrics and test builds are new to the team	**Launch-blocking**	Build it first, as its own phase, with a ready-made checklist; flag any overrun at the week 4 gate
T4	App store review delays the beta	Finance apps get extra privacy checks before external testing	**Launch-blocking**	Prepare privacy labels, a delete-my-data option and a reviewer test account by week 13
T5	The 5-minute lock annoys people	Security asks for it; ADHD design asks for speed	Fast-follow	Fast face unlock that returns people to the exact screen; watch complaints in the alpha
T6	Testers cannot link real banks	Plaid's real-bank access needs approval and has costs; the beta uses test banks	Fast-follow	The beta runs on manual mode; apply for real-bank access during the beta

#	Risk	Why we think it is real	Urgency	What we do about it
T7	People turn notifications off	Many users deny notification permission	Track	Bills also count down inside the app; ask for permission only when a reminder is switched on

Paper tigers

- **"PocketGuard already shows a left-to-spend number."** True, but our difference is the fresh start, calm design and the purchase pause. The number is expected, not our edge.
- **"No one will use it without linking a bank."** Many people do not trust bank linking. Manual mode serves them, and the beta will show how many link later.
- **"Encryption will make the app slow."** The amount of data is small; encrypted storage is not noticeably slower at this size.
- **"It needs charts and reports to look like a real finance app."** Our research says busy dashboards overwhelm this audience. Reports stay out of the MVP.

Elephants

- **Are we giving financial advice by accident?** A phrase like "you can afford this" could read as advice. *Check:* review all wording against "budgeting tools only" before the beta.
- **What if the number is wrong?** If people forget to log cash, safe-to-spend drifts and trust drops. *Check:* in the alpha, compare the app's number with real balances each week.
- **Who runs the app after the course?** Real people's data needs ongoing care, support and hosting costs. *Check:* decide before inviting testers.
- **Is the name final?** "FocusFinance" is not confirmed. *Check:* decide and check the name is free before store listings.
- **Who exactly is it for?** Should autistic adults and other neurodivergent people be included? *Check:* decide before recruiting starts.

Action plans for launch-blocking risks

Risk	Action	Owner	Due
T1 Scope	Agree phase gates and the cut order with the instructor	Product manager	Week 1
T2 Testers	Open a waitlist; post in 3 ADHD communities; reach 150 sign-ups with a balanced gender mix	Product manager	Start week 6, done week 12
T3 Security	Finish the Phase 1 security checklist before any feature work	Mobile developer	Week 4
T4 Store review	Privacy labels, delete-my-data, reviewer account, submit beta build	Product manager and developer	Week 13

We revisit this list at the week 12 gate, three weeks before the beta.

Decisions needed from stakeholders

Decision	Options	Recommendation	Needed by
When does the beta start?	(a) Week 15 with the full scope; (b) keep an earlier date and ship Phase 5 (bank linking, widget) as a beta update	**(a)**, unless the date matters more than bank linking; then (b)	Week 1
Is "FocusFinance" the product name?	Keep it, or choose another	Decide early; the name appears on the store listing and invites	Week 12
How long before the app locks itself?	5 minutes (requirements document), or longer	Keep 5 minutes; review with alpha feedback	Week 7
Who is the beta for?	Adults with ADHD only, or neurodivergent adults more broadly	ADHD first, so the results are clear; widen after the beta	Week 6
Who looks after the app and its data after the course?	Team member, instructor programme, or end the beta with data deleted	Agree before any real person's data is stored	Week 13

How we know a phase is done

A phase is finished only when all of these are true. We show them in a short demo at each gate.

- [ ] It works on a real phone, not just on a computer.
- [ ] Automated tests pass, and the "what if it goes wrong" cases were written first.
- [ ] Every new screen has its four states and passes the ADHD design checklist.
- [ ] The security checklist passes: money data is only ever stored encrypted.
- [ ] It works with a screen reader, with the largest text size, and with reduced motion.
- [ ] The phase's gate (in the timeline table) is met.

PART 2

Technical Requirements

ADHD Budgeting App: Technical Requirements Document

October 26, 2026 · Live version: Claude Doc · Companion: MVP Build Plan

Purpose and how to read this document

This document says **what the app must do and how it must behave** so it can be built, tested and trusted. It sits beside the *MVP Build Plan* (which says when and in what order) and the security requirements in BANKING_PRD_TECHNICAL_REQUIREMENTS.md.

Who it is for:

- **Stakeholders:** read the plain-language column in each table and skip the technical names in brackets.
- **Designers:** the functional requirements and the design section list the screens, states and rules you design for.
- **Developers:** every requirement has an ID (for example **FR-03** or **SEC-02**) to reference in tasks, tests and code reviews.

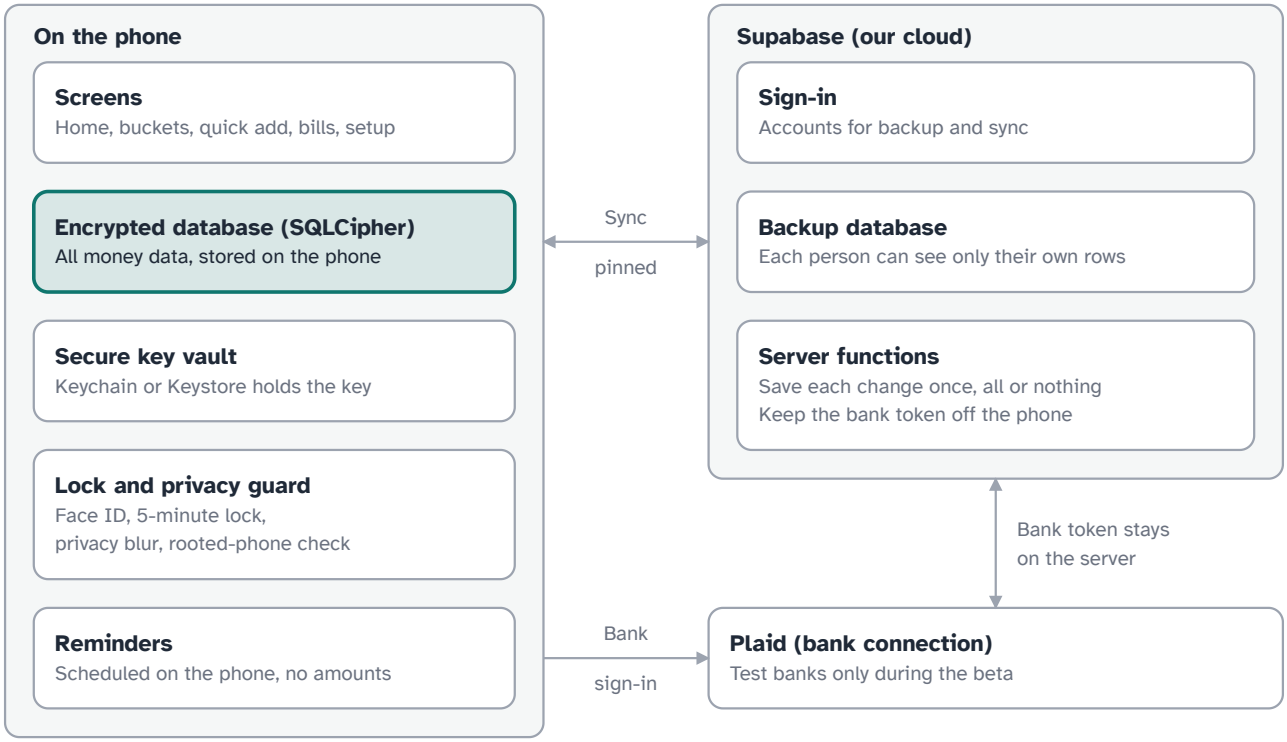
Requirements use **must** for things the beta cannot ship without and **should** for things we aim for but could defer.

How the system fits together

The app is **local-first**: everything a person needs works on their phone, even with no internet. The cloud is used for sign-in, backup and bank linking.

Money data lives encrypted on the phone; the cloud is a backup

Bank details never reach the phone: Plaid gives a token to our server instead



- **On the phone:** the screens, an encrypted database holding all money data, a secure vault (built into iPhone and Android) holding the key to that database, the lock and privacy guard, and reminders.
- **Supabase (our cloud):** accounts, a backup copy of each person's data that only they can see, and small server functions that save every change exactly once.
- **Plaid:** connects to banks. The person signs in to their bank inside Plaid's own screen; our server, not the phone, keeps the resulting bank token.
- **Connections:** all traffic uses encrypted HTTPS, and the app only talks to servers whose identity it recognises ("pinned").

Technology choices

Part	Choice (technical name)	Why, in plain words
App	Expo (React Native) with TypeScript	One app for iPhone and Android; TypeScript catches mistakes before users see them

Part	Choice (technical name)	Why, in plain words
Screens and navigation	Expo Router	Screens are organised like folders; supports pop-up sheets for quick add
Test versions on phones	Development build (expo-dev-client), built with EAS	The security features need a real build; the simpler Expo Go app cannot run them
Storage on the phone	expo-sqlite with SQLCipher (AES-256 encryption)	Works with no internet; scrambled so a lost phone does not expose money data
Key storage	expo-secure-store (iPhone Keychain, Android Keystore)	The phone's built-in safe for the database key
Unlock	expo-local-authentication	Face ID or fingerprint, with the phone passcode as a fallback
Privacy cover	expo-blur	Hides amounts in the app switcher
Unsafe phone check	jail-monkey	Spots phones whose built-in protections were removed
Trusted connections	react-native-ssl-public-key-pinning	The app talks only to our real servers
Animation	Reanimated and Lottie	Short, calm confirmations that switch off with reduced motion
Reminders	expo-notifications (scheduled on the phone)	Reminders work without our servers
Cloud	Supabase: sign-in, Postgres database with row-level security, Edge Functions	Backup, sync and safe saving, with each person locked to their own data
Bank data	Plaid (test mode during the beta)	Trusted bank connection used by most budgeting apps
Automated tests	Jest and React Native Testing Library	Checks behaviour the way a user experiences it
Release hardening	Hermes engine, R8 code shrinking, logs removed	Makes the app harder to pick apart and keeps data out of logs

Functional requirements

What each feature **must** do. "Done when" is how testers and stakeholders check it. Phase numbers match the *MVP Build Plan*.

ID	Feature	The app must	Done when	Phase
FR-01	Safe-to-spend	Show the safe-to-spend number first on the home screen; a tap shows how it was worked out	Number = balance − unpaid bills due before payday (or the next 30 days) − savings set aside. Below zero shows $0 and a calm note, never a red negative	2
FR-02	Visual buckets	Show Spending, Bills and Savings as buckets; let people move money between Spending and Savings	Buckets always add up to the balance; a move counts once even if tapped twice, and can be undone	2

ID	Feature	The app must	Done when	Phase
FR-03	Quick add	Log a cash expense in 2 taps; category optional	Two taps from home; an undo option appears after every add and delete	2
FR-04	Bill countdown	List bills due in the next 7 days, soonest first, with days left	"Due today" for day 0; marking a repeating bill paid moves it to its next date	2
FR-05	Manual mode	Let people type a balance instead of linking a bank	Every feature except import works without a bank; linking can happen later in settings	2
FR-06	Goal picker	Offer 3 simple goals	One tap to choose; the home screen order follows the goal	3
FR-07	Default categories	Provide 5 starter categories (bills, food, fun, transport, other)	None are required during setup	3
FR-08	Step progress and autosave	Show "Step N of 4" and save after every step	Closing the app, or the automatic lock, returns the person to the same step	3
FR-09	Fresh start	After 7+ days away, offer a one-tap restart	Old uncategorised items move to "other" and the week restarts; the app never says how long the person was away	4
FR-10	Weekly check-in	Run a 60-second review on a day and time the person chooses	Celebrated when done; no streaks anywhere	4
FR-11	Reminders	Remind about bills at a chosen lead time	Snooze (1 hour, tomorrow), mark paid, mute; quiet hours 21:00 to 08:00; no amounts unless switched on	4
FR-12	Pause before purchase	Optional prompt above an amount the person sets	Offers a 24-hour wishlist instead of buying now	4
FR-13	Focus Mode	Hide everything except safe-to-spend and quick add	Motion and sound can be switched off	4
FR-14	Transaction import	Import and sort bank transactions	Each is given a category; a corrected category is remembered for that merchant	5
FR-15	Bill detection	Suggest likely repeating bills from imported transactions	Same merchant, 2+ times, amounts within 10%, about weekly, fortnightly, monthly or yearly; the person confirms each, nothing added silently	5
FR-16	Home-screen widget	Give a shortcut to safe-to-spend and quick add	Shows "Tap to view" by default; the amount appears only if the person opts in	5
FR-17	Account and data control	Offer sign-in for backup and a "delete my data" option	Deleting removes the cloud copy and the data on the phone	5

Data: what we store and how it is protected

Data	Where it lives	How it is protected
Balances, expenses, bills, buckets, categories	The phone's encrypted database; a backup copy in Supabase if the person signs in	Encrypted on the phone; in the cloud each person can see only their own rows; sent only over HTTPS
Setup progress and unfinished entries	The phone's encrypted database	Encrypted, so the lock never loses someone's place

Data	Where it lives	How it is protected
Database key	The phone's secure vault	Never leaves the phone and is never backed up
Sign-in session	The phone's secure vault	Cleared when the person signs out
Plaid bank token	Our server only	Never sent to the phone
Display preferences (theme, reduced motion)	Simple phone storage	Not sensitive, so no encryption needed
Usage events	Analytics	Event names only ("finished setup"); never amounts, merchants or balances

We never store: bank passwords (Plaid handles them), full account or card numbers, or government ID numbers.

Rules for every developer:

- Money is stored as whole cents (1250 = $12.50) so totals are never off by a fraction of a cent.
- Each record gets a unique ID created on the phone, which also stops it being saved twice.
- Logs and error reports never contain personal or money data.

Security and privacy requirements

All of these are **must** for the beta. The security foundation is built first (Phase 1) and reviewed again before launch (Phase 6).

ID	Requirement	In plain words
SEC-01	Encrypt the phone's database with SQLCipher (AES-256); keep the key in Keychain or Keystore	A lost or stolen phone does not reveal anyone's money
SEC-02	No financial or personal data in simple storage (AsyncStorage), logs, web addresses or notification contents	Sensitive data lives in one protected place only
SEC-03	Unlock with Face ID or fingerprint, with the phone passcode as a fallback	Only the owner opens the app; nobody gets stuck
SEC-04	Lock after 5 minutes without use; clear sensitive data from memory; return to the same screen after unlock	A phone left on a table is safe, and nothing is lost
SEC-05	Drop a blur over the screen the moment the app becomes inactive or goes to the background; keep it up until the person is back in the app, then fade it out (instantly with reduced motion)	Amounts never show in the app switcher, after waking the phone, or to someone looking over a shoulder
SEC-06	Block rooted or jailbroken phones with a calm explanation screen	Phones with removed protections cannot keep data safe
SEC-07	HTTPS only; TLS 1.3 on our servers; certificate pinning with a backup pin	Nobody can listen in or pretend to be our server
SEC-08	No `console` logging in release builds; code shrinking and obfuscation (R8, Hermes)	The released app gives away as little as possible

ID	Requirement	In plain words
SEC-09	Every save that changes money carries a unique key (`X-Idempotency-Key`)	A double tap, a triple tap or a retried save counts only once; the button ignores extra taps until the save finishes
SEC-10	Saves that change several things happen in one transaction that rolls back on failure	Money is never half-moved
SEC-11	No secrets in the app or the repository; the Plaid secret lives only on the server	Keys cannot be copied out of the app
SEC-12	Row-level security on every cloud table	Each person can reach only their own data
SEC-13	Widget, notifications and app switcher show no amounts unless the person opts in	Money stays private on a lock screen or a shared desk
SEC-14	Never store or show full account numbers: keep Plaid's account tokens and show only the last 4 digits	A screenshot or a stolen copy reveals no usable account details

Design, accessibility and wording requirements

Our users find planning, memory and focus hard, and money needs all three. These rules keep every screen low-effort and free of shame.

ID	Requirement	Why
UX-01	Every screen that loads or saves data is designed in four states: Initial, Loading, Success and Error	Nobody is left staring at a blank or broken screen
UX-02	One decision per screen; secondary totals and details stay hidden until tapped, never combined into one busy feed	Fewer choices means less overwhelm
UX-03	Body text at least 16 point, Atkinson Hyperlegible or Lato, line height about 1.5; helper text never italic	Easy to read when attention is low
UX-04	Contrast at least WCAG AA (4.5 to 1); light and dark mode	Readable for everyone, in any light
UX-05	Touch targets at least 44 × 44 point	Easy to hit when rushing
UX-06	Colour always comes with a word or icon (teal Spending, indigo Bills)	Works for colour-blind users
UX-07	One short helper sentence beside a control the first times it matters, which can be hidden later	Coaching exactly when it is needed
UX-08	Success animations under 1 second; replaced by a still message when reduced motion is on	Small wins without overstimulation
UX-09	Shame-free wording (examples below); no red alarms, no streaks	Slips should not push people away
UX-10	Every control has a screen-reader label; amounts are read as words ("120 dollars")	Works with VoiceOver and TalkBack
UX-11	Respects the phone's text size up to the largest setting	Nothing is cut off for people who need big text

ID	Requirement	Why
UX-12	No pop-ups in the middle of a task; buttons say what they do ("Add expense", not "Next")	Keeps people moving
UX-13	Loading shows calm, low-contrast skeleton placeholders that pulse slowly (about once a second), never fast spinners; the placeholders stay still with reduced motion	Waiting does not add visual stress
UX-14	Errors use soft sage or blue, plain words and one next step, and say the person's money is safe; never error codes, capitals or red blocks	A problem feels fixable, not alarming

Instead of	We write
"You're over budget"	"Here's where things landed this week"
"You missed 12 days"	"Welcome back. Want a fresh start?"
"Session expired"	"Locked to keep your money private. Unlock to pick up where you left off."
"Network error"	"We couldn't save that just now. Your entry is safe; we'll try again."
"Error: invalid input"	"That amount needs a number, like 12.50"

Speed, reliability and offline

ID	Requirement	Level
PERF-01	Checking safe-to-spend, from unlocking to seeing the number, takes under 5 seconds	Must
PERF-02	The app reaches the unlock screen in under 2 seconds on a mid-range phone	Should
PERF-03	Quick add saves in under 1 second	Should
PERF-04	Lists scroll smoothly with 1,000 transactions	Should
REL-01	Everything except bank import and sync works with no internet; changes sync when the phone is back online	Must
REL-02	Nothing is lost or half-saved if the app closes or the phone switches off mid-save	Must
REL-03	Retrying a save never creates a duplicate	Must
REL-04	At least 99% of beta sessions end without a crash; crash reports contain no personal data	Should
REL-05	Supports the current and previous major versions of iOS, and Android 10 or later (to confirm in Phase 0)	Should

Outside services

Supabase (our cloud)

- **Sign-in is optional.** It is needed only for backup and bank linking, so manual mode works without an account.
- **Database:** Postgres with row-level security on every table.

- **Saving money changes:** through Edge Functions or database functions that check the unique save key and run as one transaction (SEC-09, SEC-10). The standard Supabase data API cannot check save keys, so it is not used for money changes.

Plaid (bank connection)

1. Our server asks Plaid for a one-time link token.
2. The person signs in to their bank inside Plaid's own screen in the app.
3. Plaid returns a temporary token, which the app hands straight to our server.
4. Our server exchanges it for the long-lived bank token and keeps that token itself.
5. Our server fetches transactions and sends them to the phone over HTTPS.

The beta uses Plaid's **test mode (sandbox)** with practice banks. Real banks need Plaid's approval; we apply during the beta.

Reminders

- Scheduled on the phone (local notifications), so they work without our servers.
- Permission is asked only when someone first turns on a reminder, never at first launch.
- Notification text has no amounts by default; it carries only an ID, and the app looks up details after unlock.
- Reminders are rescheduled every time the app starts, in case the phone cleared them.

App stores

Beta builds go out through **TestFlight** (iPhone) and **Google Play internal testing** (Android).

Testing and quality checks

We test what could go wrong **before** we test the happy path: no internet, a timeout, a typo in an amount, a double tap.

Check	What it proves	When
Money-math tests	Safe-to-spend, buckets and bill dates are always correct (for example: three 10-cent items add up to exactly 30 cents; a bill after payday is not counted)	Written before the code, every change
Screen tests	Each screen works the way a person uses it, in all four states	Every change
Security tests	A double tap creates one record; a failed save rolls back; the app locks after 5 minutes; the blur appears in the background; nothing sensitive is in simple storage	Every change
Automatic checks (CI)	Type check, lint (fails on any leftover `console` log) and all tests pass on GitHub	Every push
Accessibility check	Screen reader, largest text size and reduced motion all work through setup, safe-to-spend, quick add and undo	Each phase

Check	What it proves	When
Real-phone check	Works on an actual iPhone and Android phone	Each phase
Security review	The full security checklist (SEC-01 to SEC-14) and the manual checks below	Phases 1 and 6
Beta feedback	In-app feedback, short surveys, crash reports	Throughout the beta

Manual security checks at every phase gate

A person runs these four checks on a real phone, because automated tests cannot see what is on screen.

- [ ] **Privacy blur in the app switcher:** with balances showing, swipe up to the app switcher. The blur appears the moment the app shrinks.
- [ ] **Waking a locked phone:** lock the phone on a balance screen, unlock it and return to the app. The blur shows first and fades only once the person is back in the app.
- [ ] **Double-tap guard:** move $5 between buckets and tap Confirm three times quickly. Exactly one $5 move is saved.
- [ ] **Walk-away lock:** leave the app untouched on a balance screen for 5 minutes. It locks, the amounts disappear, and unlocking returns to the same screen.

Environments and release

Environment	Who uses it	Bank data	Cloud
Development	Developers	Plaid test banks	Development Supabase project
Internal alpha (from week 7)	The team	Plaid test banks; manual mode for real spending	Staging Supabase project
Closed beta (from week 15)	About 100 invited testers	Plaid test banks; manual mode for real spending	Production Supabase project, holding real people's data

Every release must: pass code review and the automatic checks, meet the build plan's definition of done, and keep keys in EAS environment variables (never in the code). Because the beta holds real people's data, the Phase 6 security review is required before the first invite goes out.

Assumptions, limits and open questions

Assumptions

- A small team (product manager, designer, one or two developers working with Claude Code).
- Free or starter tiers of Supabase and Plaid cover about 100 testers.
- Test banks are enough for the beta, because manual mode carries real spending.

Known limits

- The app cannot force TLS 1.3 on older phones. Our servers require it where possible, and certificate pinning protects older connections.
- Rooted and jailbroken checks catch most, not all, unsafe phones.
- Face ID and fingerprint depend on the phone, which is why the passcode fallback is required.
- The sample balance component in the security requirements file is not used as written: it uses 12 and 14 point italic text and stores the balance as text. We use 16 point or larger, never italic, and whole cents.

Open questions

Question	Recommendation	Decide by
Is 5 minutes the right lock time for ADHD users?	Keep 5 minutes; review alpha feedback	Week 7
Should an account be required?	No; optional, for backup and bank linking	Week 2
Minimum phone versions?	Current and previous iOS; Android 10 or later	Week 2
Which crash-reporting tool?	One that can strip personal data before sending	Week 12
What happens to beta data after the beta?	Delete it, or ask testers before keeping it	Week 13